High-Performance Order Matching Engine — Project Overview

> **46,500+ lines of C++20** | 85 headers | 13 source files | 75 test files | 395 CTest targets
>
> A C++20 low-latency matching engine drawing on institutional exchange design principles, built for sub-150ns processing latency and horizontal scalability.

---

1. Executive Summary

This is a C++20 low-latency order matching engine with institutional-grade architecture drawing on exchange design principles. It implements O(1) price-level lookup via `FlatPriceMap`, lock-free MPSC queues, thread-per-symbol horizontal scaling, CRC-32 journaling with deterministic replay, four wire protocols (FIX 4.2/4.4, OUCH 4.2, ITCH 5.0, SBE) over both real TCP and UDP transports, MoldUDP64 multicast with gap-recovery retransmission service, TLA+-verified safety invariants, cross-host log replication, and a complete operational stack including config management, webhook alerting, Prometheus metrics, and Docker deployment.

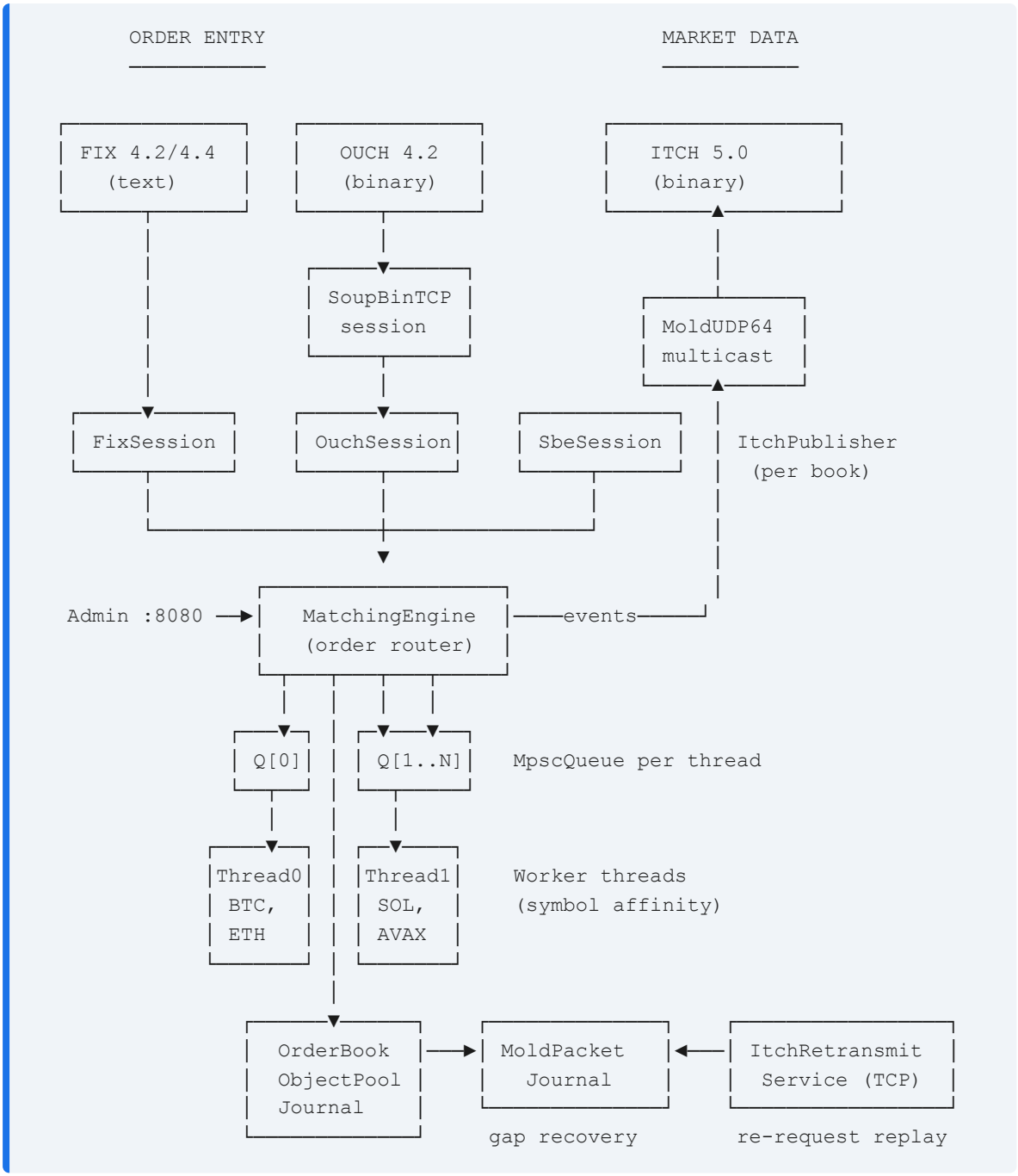
Codebase Statistics

Metric	Count
Total C++ LOC	~46,500
Header files (`include/`)	85
Source files (`src/`)	13
Test files	75
Individual test cases	395 CTest targets
TLA+ specifications	12 (454M+ states verified)
Documentation files	6 (plus architecture/benchmark docs)

---

2. Core Engine Architecture

System Topology



Order Types Supported

- **Standard**: Limit, Market
- **Time-In-Force**: IOC (Immediate-or-Cancel), FOK (Fill-or-Kill), DAY, GTD (Good-Til-Date)
- **Conditional**: Stop, StopLimit, TrailingStop
- **Institutional**: Pegged, Iceberg (hidden quantity), Hidden, PostOnly, MIT, MOC, LOC
- **Match Algorithms**: Price-Time FIFO and Pro-Rata allocation with LMM/DMM floor guarantee (40% of available qty) and rounding-remainder priority

Core Data Structures

Component	Purpose	Complexity
`FlatPriceMap`	Price-level lookup by tick index	O(1) insert/lookup
`FlatHashMap`	Robin-Hood open-addressing hash map	O(1) amortized
`IntrusiveList`	Doubly-linked order queue per price level	O(1) insert/remove
`ObjectPool`	Pre-allocated slab allocator for `Order` nodes	O(1) alloc/dealloc
`RingBuffer`	Cache-line aligned lock-free ring buffer	O(1) push/pop
`MpscQueue`	Multi-producer, single-consumer lock-free queue	O(1) push/pop

---

3. Concurrency & Networking

Thread-Per-Symbol Partitioning

N worker threads with independent lock-free `MpscQueue` ring buffers. Orders are deterministically routed by `hash(symbolId) % numThreads`, eliminating cross-thread contention on the hot path.

Multi-Protocol Order Entry

All four protocols dispatch into the same `MatchingEngine`.
- **FIX 4.2 / 4.4**: ASCII text + checksum, full session-layer state machine. `TransactTime` validation and version negotiation per accepted `BeginString`.
- **OUCH 4.2**: NASDAQ binary order entry, big-endian fixed-width.
- **SBE (Simple Binary Encoding)**: FIX-TG schema-driven binary (CME / ICE style). Forward and backward compatibility. Encode speed of ~1 ns/op.
- **SoupBinTCP**: TCP envelope wrapping OUCH on the wire.

Market Data Stack

- **ITCH 5.0 Publisher**: Converts engine events to ITCH 5.0 frames.
- **MoldUDP64 Multicast**: Batched publisher with MTU auto-flush + gap-detecting subscriber.
- **Gap Recovery**: Bounded ring of journaled MoldUDP64 messages backing a SoupBinTCP-over-TCP retransmission service.
- **Shared Memory IPC**: L2 market data distribution to co-located consumers via `shm_open`.

---

4. Regulatory & Risk Management

Feature	Implementation
Self-Match Prevention (SMP)	Cancel-Taker strategy — prevents wash trading
Circuit Breakers	Configurable % price band halts (volatility protection)
OTR Monitoring	Real-time Order-to-Trade ratio tracking per participant
Kill Switch	Instant cancellation of all orders per participant across all symbols
Pre-Trade Risk Limits	Max order size, notional value, and position limits per participant
Rate Limiting	Token-bucket throttling at ingress (configurable rate + burst); `RateLimiter::reconfigure()` live-updates default rate/burst and clears all participant buckets — called on SIGHUP
Queue Backpressure	Rejects orders when queue exceeds configurable threshold (default 80%)
LMM/DMM Privileges	`ParticipantRole` enum; ProRata 40% floor guarantee to LMM/DMM orders; remainder priority

5. Microstructure Research Infrastructure

A self-contained quantitative research layer that runs against the live matching engine, enabling empirical microstructure analysis and strategy development.

Component	Purpose
`SimulationDriver`	Multi-agent market simulator (NoiseTrader, MarketMaker, InformedTrader)
`MicrostructureMetrics` / `ResearchHarness`	Snapshot collection and research session management
`VPINCalculator`	Volume-Synchronized Probability of Informed Trading
`SpreadDecomposition`	Huang-Stoll and Glosten-Harris decomposition (adverse selection, inventory, order processing components)
`OrderFlowAnalytics`	Roll spread, queue imbalance signal, logistic fill-probability model
`ImpactModel`	Almgren-Chriss + square-root market impact laws
`ExecutionAnalytics` / `OptimalExecution`	Almgren-Chriss execution schedule, arrival-price and VWAP slippage

Component	Purpose
`CalibrationPipeline`	Nelder-Mead minimization of spread decomposition residuals
`BacktestEngine`	Historical tape replay against research models
`SignalGenerator`	Multi-factor composite signal (momentum, spread, VPIN, imbalance)
`PaperTrader`	Signal-driven IOC order submission with position and P&L tracking
`ResearchDashboard` / `ResearchSerializer`	Result visualization and persistence

---

6. Reliability & High Availability

CRC-32 Journaling

- Write-ahead log with atomic CRC-32 integrity on every entry.
- Crash recovery: replay stops at first corrupted/truncated record.
- Atomic checkpoint: snapshots active book state, rewrites journal.
- Deterministic replay via virtual clock (`setExpiryClock(ClockFn)`).
- Auto-rotation: `OB_JOURNAL_MAX_SIZE_MB` env var triggers `engine.checkpoint()` from the main loop when `Journal::needsCheckpoint()` fires.

Cross-Host Log Replication (wired end-to-end)

- `ReplicationCoordinator` — TCP log-shipping from primary to backup, instantiated in `src/main.cpp` driven by `OB_NODE_ROLE` / `OB_PRIMARY_HOST` / `OB_JOURNAL_PATH` env vars.
- `Journal::onCommit` hook ships each fsync-durable batch to the backup; backup's `applyReplicatedEntry` writes to its own journal so the replica is durable across its own restarts.
- `HeartbeatMonitor` — configurable failure detection timeout.
- `LeaderLease` + `LeaseGrant` propagation — primary broadcasts lease state every heartbeat tick; backup's `tryAcquire` is fenced on local lease expiry, not just heartbeat miss. Prevents split brain under partial network failure (verified by 19-scenario chaos suite: partition, asymmetric partition, 30% packet loss, +30s clock skew — 0 split brain in all).
- Transport auto-reconnect — `receiveLoop` re-runs `connectTo()` against saved host/port after socket loss; ~3 ms recovery in chaos tests. Reconnect uses exponential backoff: 500ms → 30s cap, resets to base on successful connect.

- Snapshot catchup on join — primary streams every resting order via `MatchingEngine::streamSnapshot` when a backup connects; idempotent on receiver.
- Automatic backup promotion on combined heartbeat-miss + lease-expiry; `setReplayModeAllBooks(false)` flips backup into a live primary.

---

7. Observability & Operations

Observability Features

- **Structured Logging**: Pluggable `StructuredSink` API (Null, JSON Stderr, Capturing). 7 typed `obSink().log()` events wired in `OrderBook.cpp` (accepted/cancelled/rejected/fill/breaker/state); backends hot-swappable via typed helpers in `StructuredLog.h`.
- **Prometheus Metrics**: `MetricsRegistry` with `/prometheus` text-exposition endpoint. Tracks order flow, queue depth, latencies, and journal health.
- **Webhook Alerting**: `AlertDispatcher` with targets for Slack, PagerDuty, or HTTP webhooks.

Admin HTTP Server (Port 8080)

Endpoint	Purpose
`GET /health`	K8s liveness probe
`GET /readyz`	K8s readiness probe — HTTP 503 until warmup, then 200; auth-exempt; separate from liveness `/health`
`GET /metrics`	Internal counters (JSON)
`GET /prometheus`	Prometheus text exposition
`GET /book?symbolId=0`	L2 order book snapshot
`GET /otr?participantId=1`	Order-to-trade ratio

---

8. Performance Benchmarks

Measured using `HonestBenchmark` — a single deterministic order flow (50K orders, seed=42) fed through three paths on Apple Silicon ARM64, Clang C++20 -O2.

Three-Path Latency (identical order flow)

Path	What's Included	P50	P99	P99.9	Throughput
Core matching	OrderBook + STP + WashTrade + LULD	**125 ns**	333 ns	458 ns	6.6M ops/s
Engine wrapper	+ sequence alloc, rate limiter	84 ns	292 ns	417 ns	7.1M ops/s
Full-stack journal	+ GroupCommit (batch=64, fdatasync)	1,040 ns	2.7 ms	4.0 ms	22K ops/s

Binary Codec Microbenchmark

Codec	Message	ns/op	M ops/s
OUCH 4.2	EnterOrder encode (49B)	112.0	8.9
ITCH 5.0	AddOrder encode (36B)	34.1	29.4
SBE	**NewOrderV1 encode (32B)**	**1.0**	**1015**
SBE	**NewOrderV1 decode (32B)**	**0.5**	**2128**

---

9. Verification & Testing

Test Suite — 75 Executables, 395 CTest Targets

The testing infrastructure includes Unit, Functional, Integration, Chaos, Property, Shadow, and Benchmark testing categories across 75 test executables and 395 CTest targets. Key mechanisms:
- **Shadow Mode**: Dual-book divergence detection, validating FIFO compliance.
- **Fault Injection**: 10+ injection points (short-writes, pool exhaustion, EAGAIN injection) with zero-cost overhead in production.
- **Coverage-Guided Fuzzing**: libFuzzer harness for protocol parsing and order flow.
- **Sanitizers**: ASan, UBSan, and TSan checks integrated into CI/CD.

Formal Verification (TLA+)

12 TLA+ specifications, model-checked with TLC:
- `MatchingEngine.tla` : 454M states verified. Zero violations for `NoNegativeQuantity` , `FIFO_Preservation` , and `GTD_Expiry_Correctness` .
- `Replication.tla` : Realistic lease-propagation model (no god-mode `~primaryAlive` guard on `BackupPromote` ; promotion requires both heartbeat-miss AND local-lease-expiry). 1373 → 4192 states verified at `MaxEntries=6 → 10` , zero violations. A bug-injected variant (lease check stripped) reproduces split brain in 188 states, confirming the verification is genuine.

- `MpscQueue.tla`: Lock-free ring buffer linearizability.
- `SnapshotLocked.tla`: Mutex torn-snapshot prevention.

---

10. File Structure

```
include/              85 header files — core logic and networking
src/                  13 source files — thin compilation units
tests/                75 test files, 395 CTest targets
benchmarks/           9 benchmark binaries
fuzz/                 9 files — coverage-guided fuzzing harnesses
spec/                 TLA+ formal specifications (12 specs)
tools/                CLI tools and data utilities
config/               Example configuration files
docs/                 Runbooks, CapacityPlanning, ProductionReadiness
```

---

11. Remaining Work

The system is architecturally complete. The main remaining gaps require specialized hardware not typically available in standard environments:
- **x86 Bare Metal Benchmarks**: Needs multi-socket EC2 c5.metal instance.
- **Kernel Bypass Networking**: DPDK and io_uring seams are prepared, but require specific NICs (e.g., Solarflare/Onload) and tuning.
- `Replication.tla` **Verification**: ✅ done — see Verification section above. The original "not yet verified" footnote is obsolete.
- **Wire-to-Wire Latency Validation**: Requires a multi-host test rig with hardware timestamping.

Developed for professional quantitative trading systems.
C++20 · ~46,500 LOC · 75 test executables · 395 CTest targets · 19 multi-container chaos scenarios · 454M TLA+ states verified on MatchingEngine.tla · 12 TLA+ specifications